

← Go Back

Hardware

Portenta X8

Tutorials

01. Portenta X8 User Manual

- 02. Portenta X8 Fundamentals
- 03. Uploading Sketches to the M4 Core on Arduino Portenta X8
- 04. Data Exchange Between Python® on Linux & Arduino Sketch
- 05. Managing Containers with Docker on Portenta X8
- 06. Using FoundriesFactory® Waves Fleet Management
- 07. Deploy a Custom Container with Portenta X8 Manager
- 08. How To Build a Custom Image for Your Portenta X8
- 09. How To Update Your Portenta X8
- 10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana
- 11. Output WebGL Content on a Screen
- 12. Multi-Protocol Gateway With Portenta X8 & Max Carrier
- 13. Running WordPress & Database Containers on Portenta X8
- 14. Portenta X8 Firmware

Home / Hardware / Portenta X8 / 01. Portenta X8 User Manual

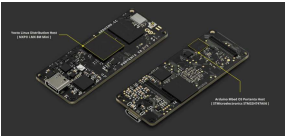
01. Portenta X8 User Manual

Get a general overview of Portenta X8 and its features.

Author · Marta Barbero · Last revision · 02/28/2025

Introduction

Portenta X8 is a powerful, industrial-grade System on a Module with Linux OS preloaded onboard that can run device-independent software thanks to its modular container architecture.



Portenta X8

In this user manual, we will go through the foundations of the Portenta X8 to help you understand how the board works and how you can benefit from its advanced features.

Required Hardware

- Portenta X8 (x1)
- USB-C® cable (USB-C® to USB-A cable) (x1)

ON THIS PAGE

Introduction

- Required Hardware
- Required Software
- Product Overview +
- First Use Of Your Portenta X8+
- Portenta X8 OS Image Update
- Portenta X8 with Arduino Cloud
- Portenta X8 Board Manager
- Working with Linux +
- Working With Arduino +
- Working With Portenta X8 Board Manager +
- Pins
- Communication +
- Support +
- Trademark Acknowledgments

Required Software

For Linux programming, leverage the latest Linux image available, and check [this section](#) to verify if your Portenta X8 has already been updated.

For Arduino programming, leverage [Arduino IDE 1.8.10+](#), [Arduino IDE 2](#), or [Arduino Web Editor](#), and the latest **Arduino Mbed OS Portenta Boards** Core > 3.0.1.

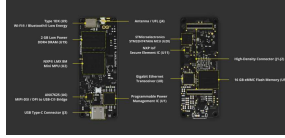
Product Overview

The Portenta X8 combines the best of two approaches, offering the flexibility of Linux and the capability for real-time applications within the Arduino environment. Developers can execute real-time tasks while simultaneously handling high-performance processing on Linux cores.

So, let's have a look at its technical specifications.

Board Architecture Overview

Portenta X8 is a powerful, industrial-grade System-on-Module (SoM) that combines a Yocto Linux distribution with the well-known Arduino environment.



Portenta X8 Tech Specs

As reported in the image above, Portenta X8 features two powerful computing units:

NXP® i.MX 8M Mini

Cortex®-A53 quad-core up to 1.8GHz per core + 1x Cortex®-M4 up to 400 MHz. This microprocessor is the one where the Yocto Linux distribution is running together with Docker containers (check [this section](#) of this user manual to learn more).

STMicroelectronics

STM32H747AI16 featuring 1x Arm® Cortex®-M7 core running up to 480 MHz and 1x Arm® Cortex®-M4 core running up to 240 MHz. This microcontroller hosts the "[Arduino Mbed OS Portenta Boards](#)" Core. M4 core is accessible and programmable by the user, while M7 is dedicated to establishing and guaranteeing the communication between i.MX 8M Mini and M4, as well as to manage peripherals through RPC. For more details, refer to [this section](#) of the user manual.

which are summarized in the table below.

NXP® i.MX 8M Mini	STMicroelectronics STM32H747AI16
Running Yocto Linux distribution with Docker containers	Running Arduino sketches with the Mbed OS Portenta Boards Core
Dedicated to high level tasks	Dedicated to real-time tasks
Manage network connectivity	No direct connection to any network stack
Manage network-based buses (e.g., Modbus TCP, etc.)	Manage field buses (e.g., Modbus RTU, CANbus, etc.)
Access to peripherals without concurrent access control	Access to peripherals without concurrent access control

In addition to the above features, here is an overview of the board's main components:

External memory: The board features an onboard 16 GB eMMC Flash memory and 2 GB Low Power DDR4 DRAM.

Wireless connectivity: The board supports 2.4 GHz Wi-Fi® (802.11 b/g/n) and Bluetooth® 5.1, provided

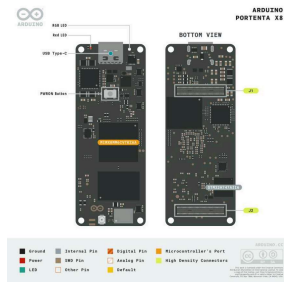
performance Wi-Fi® and Bluetooth® module allows the Portenta X8 to communicate wirelessly with other devices and systems.

Ethernet connectivity: The board features an onboard, high-performance 1 Gbps Ethernet transceiver accessible through its High-Density connectors.

Secure element: The board includes a ready-to-use secure element, the SE050C2 from NXP®, tailored for IoT devices and offering advanced security features. This allows Portenta X8 to achieve PSA certification from ARM®. For more information, click [here](#).

USB connectivity: The board features a USB-C port for power and data, which is also accessible through the board's High-Density connectors. This port is linked to a MIPI to USB-C/DisplayPort converter, enabling video output through the USB-C connection.

Power management: The Portenta X8 includes a power management integrated circuit (PMIC) to meet the demands of always-connected IoT devices.



Portenta X8 Pinout

The full pinout is available and downloadable as PDF from the link below:

[Portenta X8 Pinout](#)

Datasheet

The full datasheet is available and downloadable as PDF from the link below:

[Portenta X8 Datasheet](#)

Schematics

The full schematics are available and downloadable as PDF from the link below:

[Portenta X8 Schematics](#)

STEP Files

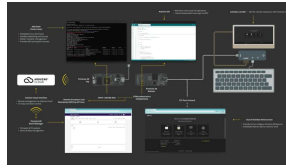
The full *STEP* files are available and downloadable from the link below:

[Portenta X8 STEP files](#)

Features Overview

Portenta X8 integrates two main

Yocto Linux and popular **Arduino** environments.



Portenta X8 Features
Overview

To explore specific sections in more detail, please click on the links below that interest you:

[Linux Environment](#)

[Arduino Environment](#)

[Arduino Cloud](#)

Portenta X8: Linux & Arduino Interoperability

The Portenta X8 runs on a **Yocto-based Linux distribution** with Docker container support, offering robust capabilities for deploying and managing device independent applications. In addition, it allows users to upload Arduino sketches to the M4 core of the STM32H7 using the Arduino IDE.

This integration leverages a service called

`monitor-m4-elf-file.service`,

which monitors for updated sketches and flashes the M4 core using *OpenOCD*, ensuring seamless interaction between Linux and Arduino environments.

The Portenta X8 also supports **Remote Procedure Call (RPC)** communication between its Linux

MessagePack-RPC to efficiently exchange data.

For more details on Portenta X8's Linux capabilities, Docker support, and Arduino integration, please refer to the **Portenta X8 Fundamentals documentation**.

First Use Of Your Portenta X8

You can now start interacting with your Portenta X8. Portenta X8 has an embedded configuration console that will guide you step-by-step in configuring your board.

Power The Board

Connect the Portenta X8 to your PC via a **USB-C® cable (either USB-C® to USB-A)**.

Once connected, you will see the Portenta X8 LEDs start blinking. Portenta X8 features two LEDs, a Power LED, and a Status LED, which can blink in parallel.



Portenta X8 LEDs

The table below describes the meaning and functionalities of LEDs.

LED Type	Colour	Meaning
Power LED	Red	Power ON
Status LED	White	OS booting in progress
Status LED	Blue	Linux Kernel running
Status LED	Green	Board connected to the Internet
Status LED	Red	STM32H7 LED, blinking when triggered in the IDE

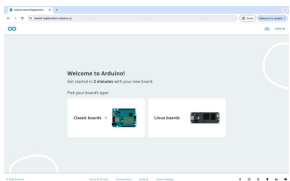
Setup with the Arduino Linux Wizard



It is recommended that you have your Portenta X8 with the latest OS version. Check [this section](#) to learn how to have your Portenta X8 up-to-date.

Once the Portenta X8 is correctly powered up, you can start interacting with it.

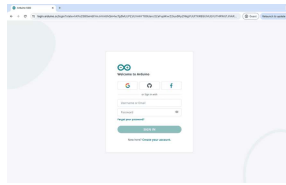
Open your browser and navigate to www.arduino.cc/start. The following page will be displayed.



First Setup

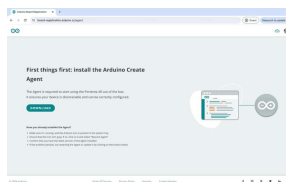
Click on **Linux boards** and sign in to your Arduino Cloud account. If

create a new one from the same webpage.



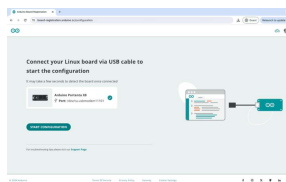
Sign into Arduino Cloud

When successfully logged in, you will be asked to download the **Arduino Create Agent** if you have not done so yet. Click on **DOWNLOAD**.



Create Agent
Installation

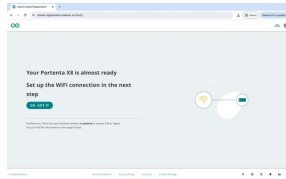
The agent will be installed on your computer. This activity might take a few minutes. Once installed, your Portenta X8 will be automatically detected by your PC after a few seconds.



Portenta X8
successfully detected

Now click on **START CONFIGURATION**. The tool will install all the required add-ons to make your Portenta X8 work efficiently with your PC via serial

You can now proceed to the setup of the board connectivity by clicking **OK, GOT IT**.



Arduino Linux Wizard Connectivity Configuration

If you face any issue with this flow or prefer to interact directly with your Portenta X8 through the command line, please refer to [this section](#) to learn how to connect with the board leveraging ADB service.

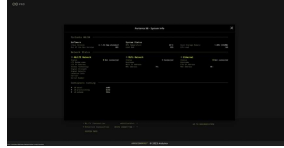
Once the setup is complete, the browser page will automatically switch to the Arduino Linux Wizard page.

The Arduino Linux Wizard page is accessible at <http://localhost:8080>. You can use this link to open the Arduino Linux Wizard dashboard anytime.



Arduino Linux Wizard Home Screen

The **SYSTEM INFO** button is located at the bottom left corner of the Arduino Linux Wizard page.



Arduino Linux Wizard System Info

This section displays details about the Portenta X8, such as hardware and software specifications, network status, and active containers.

If you disconnect the Portenta X8 and need to reopen the Arduino Linux Wizard page, use the command line and run the TCP port forwarding command:

```
1 adb forward tcp:8080 to
```

Then, access the page with <http://localhost:8080> to open the Arduino Linux Wizard dashboard.

Wi-Fi® Configuration

Click **Wi-Fi® Connection** to start configuring your network connectivity. The Portenta X8 can connect to the network through an Ethernet cable using either a USB-C® hub with an RJ45 port or a Portenta Carrier, via cellular connectivity using a Portenta Carrier and a Pro 4G Module, or through Wi-Fi®. In this tutorial, Wi-Fi® connectivity will be used.

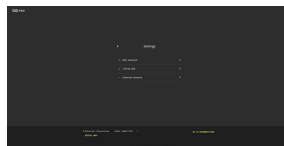


Arduino Linux Wizard Wi-Fi® Settings

 For detailed instructions on using cellular connectivity with the Portenta X8 and a Pro 4G Module, please refer to the [Cellular Connectivity with the Max Carrier & Pro 4G Module tutorial](#) or the [Cat.4 Modem section](#) of the Portenta Mid Carrier User Manual.

 For detailed instructions on using Ethernet connectivity with the Portenta X8 and the RJ45 port of the Portenta Carrier, please refer to the [Ethernet section of the Portenta Hat Carrier User Manual](#) or the [Ethernet section of the Mid Carrier User Manual](#).

Choose the **WiFi Network** option to continue network connectivity configuration with Wi-Fi® connectivity.



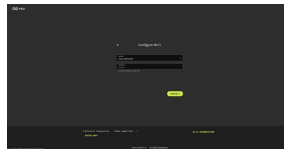
Arduino Linux Wizard Wi-Fi® Network Selection

Select your Wi-Fi® SSID. You can select a network from the



Arduino Linux Wizard
Wi-Fi® SSID set-up

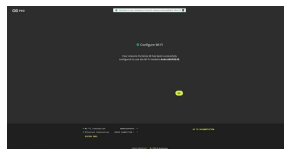
Type your Wi-Fi® password.



Arduino Linux Wizard
Wi-Fi® password set-up

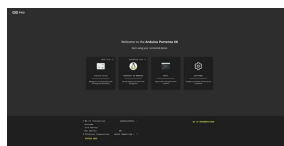
Once connected, you will get a notification confirming your Portenta X8 has connected to the selected network, and its LED will start blinking green.

Moreover, you can check the network you are connected to in the bottom left section of this dashboard.



Arduino Linux Wizard
Wi-Fi® connection
successful

Now, you can click **OK** and be redirected to the Arduino Linux Wizard homepage below.





You can change your network by clicking on the Settings button and repeating the above steps.

Arduino Linux Wizard Homepage

The Arduino Linux Wizard, hosted on the Portenta X8, allows a user to leverage the following capabilities:

Get board details

[Configure Portenta X8 Wi-Fi®](#)

[Interact with the board through the embedded Python® Alpine Shell](#)

[Provision your device to Arduino Cloud](#)

Manage the Linux distribution with the dedicated [Portenta X8 Board Manager](#)

[Update the board's operating system](#)

Portenta X8 with Python Alpine Shell

Click the **Shell** button to start using your Portenta X8 with Python-Alpine.



Arduino Linux Wizard
Shell button

This shell is running in a Python-Alpine container embedded in Portenta X8. You will find multiple examples under the directory `/root/examples` in this shell. Additionally, you can either add your own package through the command `apk add <packagename>` or start exploring the packages available online at [this link](#).



Arduino Linux Wizard
Python-Alpine Shell

Portenta X8 OS Image Update

It is recommended that you check every now and then to see if your Portenta X8 image version is up to date to have the latest security updates.

To review the image version history, refer to the [Portenta X8 firmware release notes](#).

There are four ways to update your Portenta X8 are described:

[Update for OS release V.399](#)

[Update through Arduino Linux Wizard experience](#)

[Update through Portenta X8 Manager in your](#)

account (available for all OS releases)

Update using the `uuu` tool (compatible with custom images)

You can find more details in the dedicated tutorial here: [How To Update Your Portenta X8](#).

Portenta X8 with Arduino Cloud



Note: this is an optional step. The Portenta X8 can also be used with locally without an internet connection.

Making Portenta X8 compatible with Arduino Cloud means opening many new applications. This compatibility is guaranteed by a brand-new Python container, which includes a dedicated [Arduino IoT Cloud Python library](#). Through Arduino Cloud APIs, the Python container ensures full interaction and simple porting of any Python developed application in the Arduino Cloud.

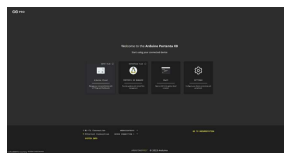


Check all the available Arduino Cloud plans [here](#) and create your Arduino Cloud account in a couple of steps (see the dedicated documentation at [this link](#)).

With the Arduino Linux Wizard experience, your Portenta X8 can be securely self-provisioned in

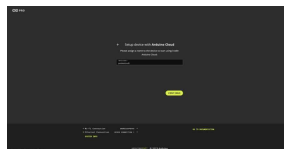
container running on X8 will do the rest. When provisioned, you can start directly interacting with an example Thing and Dashboard that will be automatically generated to guide you in this new journey.

Click the **Arduino Cloud** button to start provisioning your Portenta X8 in Arduino Cloud.



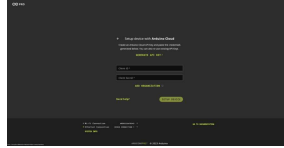
Arduino Linux Wizard
Arduino Cloud

Start setting up the device name for your Portenta X8 (in this case, *portenta-x8-test*) and click on **CONTINUE**. The same device name will be used and visualized in your Arduino Cloud space, but you can freely change it in the future.



Arduino Linux Wizard
Arduino Cloud Device
Name

At this point, you will be asked to insert your API Key credentials and Organization ID. Organization ID is optional and should be filled in only if you use a Shared Space in Arduino Cloud for Business.

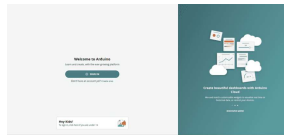


Arduino Linux Wizard Arduino Cloud API Keys

To get API keys, log into your Arduino Cloud account and select the Space you would like your X8 to be provisioned into.

Thus, click on **GENERATE API KEY** in your Arduino Linux Wizard dashboard. A new window in your web browser will allow you to log in to your Arduino Cloud space.

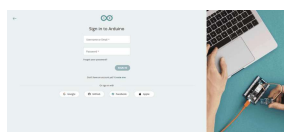
If you want to learn more about what API keys are and how they work, please take a look at the dedicated documentation available at [this link](#).



Arduino Cloud Login

Click on **SIGN IN**. If you do not have an Arduino Cloud account, create a new one from the same webpage.

Sign in to your Arduino Cloud account by adding your credentials, i.e., Username/email and Password.



You are now logged into your Arduino Cloud space. Go on by clicking on **API keys** within the account banner in the top right corner.



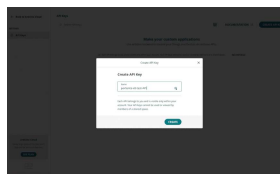
Arduino Cloud
Homepage

It is time to generate your API keys. Click on **CREATE API KEY** in the upper right-hand corner.



Arduino Cloud New API
Key

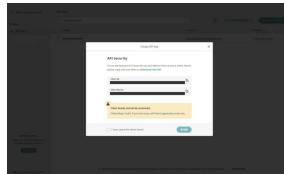
Define a name for your API key, in this case, *portenta-x8-test-API*, and click on **CREATE**. These API keys are personal and visible only from your account.



Arduino Cloud API Key
name

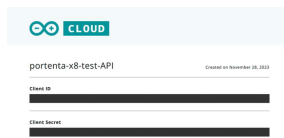
At this point, your API key has been created. Save the correspondent credentials in a safe storage space by clicking on **download the PDF**.

Keep this file safely stored; otherwise, your API credentials cannot be recovered. If you lose it, you will have to generate new API keys by repeating the above procedure.



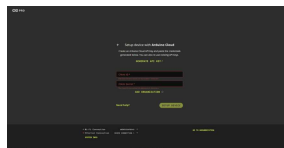
Arduino Cloud API Key

The PDF file will look like the image below and include the credentials you need to copy and paste into the Arduino Linux Wizard page.



Arduino Cloud API Key
PDF

Thus, copy the **Client ID** and the **Client Secret** credentials and paste them into your Arduino Linux Wizard dashboard as shown below.



Arduino Linux Wizard
with API Keys

If you are using an Arduino Cloud for Business account with Shared Spaces. In that case, you also need to add the Organization ID

© 2011 Blackwell Publishing Ltd *Journal of Internal Medicine* 270: 101–109



To recover the Organization ID



At this point, you can copy the

© 2011 Blackwell Publishing Ltd *Journal of Internal Medicine* 270: 103–111

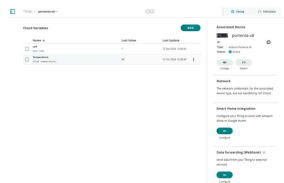


Click on **SETUP DEVICE**, and you



Arduino Linux Wizard successful Cloud provisioning

Once provisioned, the Portenta X8 will be automatically linked to an example [Thing](#) and [Dashboard](#). You can freely check them by clicking on the corresponding links embedded in the Arduino Linux Wizard.



Portenta X8 example Thing

As mentioned, Arduino provides an example dashboard that will automatically set up and be visible live after your Portenta X8 has been provisioned. To make this dashboard update its data automatically, you need to go back to your Arduino Linux Wizard and launch the example.

To do so, copying the shown code:

```
1 python3 examples/arduino
```

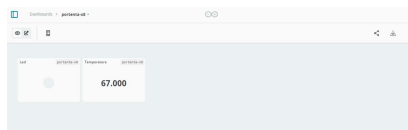
And clicking on **Launch Example** is sufficient. An Alpine-Python shell will open, and you will have

to paste the previous code here to launch the example.



Launching dashboard example

Now, you can navigate to your dashboard [here](#) to see your Portenta X8 LED blinking and the live temperature inside the microprocessor.



If you face any issues during the provisioning of your Portenta X8, feel free to repeat the procedure above.



If you would like to customize your Portenta X8 Things/Dashboards with your custom data, check [this section](#) of the user manual.

Portenta X8 Board Manager



Note: This is an optional step that enables the possibility of performing remote over-the-air updates on your Portenta X8 OS and applications. To proceed, you need an Arduino Cloud

Now, you can start connecting your Portenta X8 to the Portenta X8 Board Manager. You need an Arduino Cloud for your business account to leverage this feature.

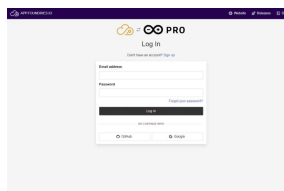
Check the Arduino Cloud for business plan with Portenta X8 Manager [here](#) and create your Arduino Cloud account in a couple of steps (see the dedicated documentation at [this link](#)).

When your Arduino Cloud for business account is correctly set up, log into it [here](#) and click on **Portenta X8 Board Manager**. The feature is located within the **Integrations** section of the Cloud.



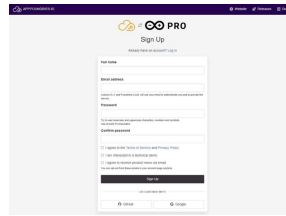
Arduino Cloud homepage with Portenta X8 Manager

At this point, you will be asked to create a new account on [Foundries.io](#) platform. It is recommended to register with the same email address you are currently using in your Arduino Cloud for business account.



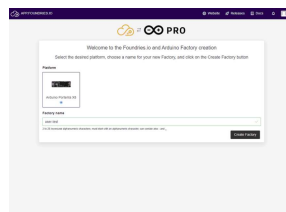
Foundries.io login

Add all your credentials and click



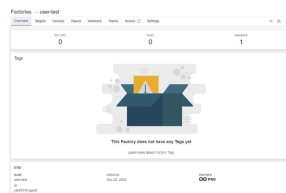
Foundries.io sign up

So, let's create your brand new FoundriesFactory. Select **Arduino Portenta X8**, define a **Factory name** for your Factory, and then click on **Create Factory**.



FoundriesFactory name

Your FoundriesFactory is correctly set up. As you can see, the Factory does not have any device connected to it.

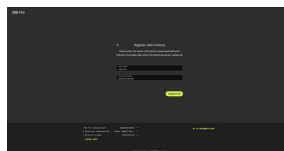
FoundriesFactory
homepage with no
devices

To provision your Portenta X8, go back to your Arduino Linux Wizard webpage and click on the **Portenta X8 Manager** button.



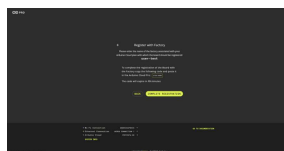
Arduino Linux Wizard Portenta X8 Manager

Enter the Factory name you have just registered, in this case, *user-test*, and assign a Board Name to your Portenta X8. This Board Name will be used to correctly identify your Portenta X8 in your FoundriesFactory. You can now click on **REGISTER**.



Arduino Linux Wizard Factory and device registration

To complete the registration of the Board with the FoundriesFactory, copy the code that appeared in your Arduino Linux Wizard.



Arduino Linux Wizard Factory code challenge

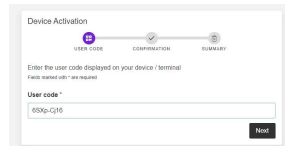
Click on **COMPLETE REGISTRATION** to be re-directed to the *Foundries.io* activation page.

Paste your token in the text box and press **Next**.

The token code is valid for **15 minutes**. If you do not use it in this time span, you will have to repeat all the above registration steps in your Arduino Linux Wizard to generate a new code.

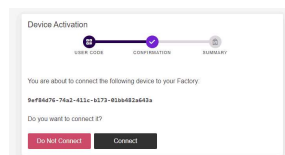


will have to repeat all the above registration steps in your Arduino Linux Wizard to generate a new code.



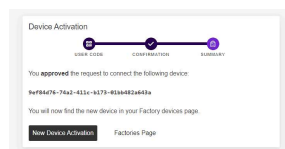
FoundriesFactory
pasted token

Confirm the addition of your Portenta X8 by pressing **Connect**.



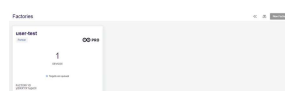
FoundriesFactory
device confirmation

Go to your FoundriesFactory by clicking on **Factories Page**.



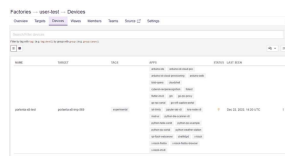
FoundriesFactory
device registration
completed

Now you will see the number of devices associated with your FoundriesFactory to be equal to 1.



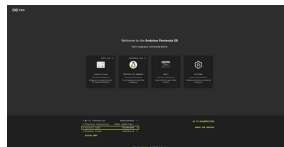
Your Portenta X8 is correctly provisioned into your FoundriesFactory.

To verify your device status, click on your FoundriesFactory, go to the **Devices** section, and check its target update and the installed containers Apps.



FoundriesFactory
device overview

You can also check within the Arduino Linux Wizard home screen for factory registration if it was successfully registered.



Arduino Linux Wizard
Factory registration
confirmation

If you want to learn more about Portenta X8 Manager features, check the  dedicated section of this user manual called [Working with Portenta X8 Board Manager](#).

Working with Linux

It is time to start interacting with the Linux OS embedded in your

and look for **ADB** inside the directory

Arduino15/packages/arduino/tools/adb/32.0.0. The Arduino15 folder may have a different location depending on your Operating System. Check [this article](#) to learn where your Arduino15 folder is located.

Android Debug Bridge (ADB) is a tool included in the SDK software (Software Development Kit) and used, among other things, to make an Android device and a computer communicate. To check if ADB is working correctly, you can type `adb devices`. Your Portenta X8 will be listed there.



Connection with ADB

If you need to install ADB, you can also download the  right tool for your Operating System directly from the [official Android website](#).

If you want to start the embedded Arduino Linux Wizard from the command line, you can continue typing in your terminal:

```
1  adb forward tcp:8080 tc
```

With this command, ADB allows you to forward the requests of your computer's

which, in this case, is the device with the name *Portenta X8*.



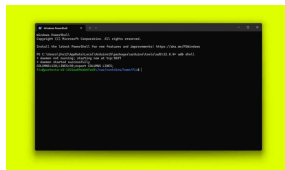
ADB forward command

Now you can open your browser, go to <http://localhost:8080> and the same Arduino Linux Wizard dashboard will appear to allow you to configure your Portenta X8.



Arduino Linux Wizard
Homepage

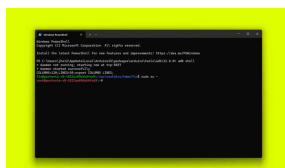
Now, you can type `adb shell` to start communicating with your Portenta X8.



ADB shell command

As it is a Linux device, you can create files, change directories, etc.

To gain admin (root) access, type `sudo su -`. The terminal prefix should turn red.



ADB shell with admin access

You can now freely program your Portenta X8 Linux OS. In the sections below, you can check out some basic commands to get started.

Manage Your Network Via CLI

To connect to a Wi-Fi® Access Point via CLI, you can use the network manager tool **nmcli**. These are some of the most used commands:

```
nmcli device wifi  
connect <SSID> password  
<PASSWORD>
```

to connect to a specific SSID

```
nmcli de
```

 to check the connection status

```
nmcli connection show
```

 to visualize the active network interfaces (and their types) on your Portenta X8

Accessing Over SSH Session

Establishing communication with the Portenta X8 via an SSH session is possible. To do so, a network connection is needed, either over Wi-Fi®, cellular or Ethernet. Using a device with DHCP server capabilities, such as a network router, is recommended for Ethernet connections. After setting up the network connection and DHCP, the Portenta X8 will be ready for

For Windows users, it is necessary to install a service tool to ease the following procedures. While Bonjour, Apple's implementation of zero-configuration networking, comes built into macOS, it is not natively included in Windows and must be installed separately.

Before proceeding on **Windows**, please install  **Bonjour Print Services for Windows** before continuing the following steps.

For macOS and Linux users, *Bonjour* is pre-installed on macOS, and *Avahi-Browse* is typically available on Linux by default. Thus, additional installation steps may be unnecessary for these operating systems.

In the subsequent sections, we will first walk you through the process on Windows, followed by details and instructions for Linux and macOS.

Using the Terminal



SSH Services
Availability Discovery

The command below is used to browse for SSH services on the local network that are advertised over a multicast Domain Name System (mDNS). This protocol resolves hostnames to IP addresses within small networks

```
1 dns-sd -B _sftp-ssh._tcp
```

By executing this command, you can discover devices offering *SFTP services* (file transfer over SSH) without prior knowledge of their IP addresses or hostnames.

The command lists these services, indicating where an SSH connection can be established for secure file transfers or shell access, helping to ease the identification and utilization of networked devices that support this protocol.



Network Query for IPv4 and IPv6

```
1 dns-sd -G v4v6 portenta
```

The command above queries the network for the *IPv4* and *IPv6* addresses associated with the hostname

`portenta-x8-<UUID>.local`. The *UUID* can be found within the instance name listed previously or if you have accessed the Portenta X8 via an ADB shell. An example command would look as follows:

```
1 dns-sd -G v4v6 portenta
```

This command is handy for finding the IP addresses of devices such as the Portenta X8 that a DHCP server may assign dynamic IP addresses. It simplifies connecting to such devices over the network by providing their current IP addresses.



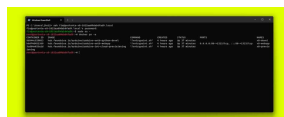
Device Ping Test

The following command sends echo requests to the device with the hostname

`portenta-x8-<UUID>.local` to check its network availability and measure round-trip time.

```
1 ping portenta-x8-<UUID>
```

This command helps verify that the Portenta X8 is online and reachable over the network and for diagnosing connectivity issues. The UUID can be ascertained by referring to the findings from an earlier SSH services scan with network query or the ADB shell.



Portenta X8

After verifying that the Portenta X8 is accessible using a simple ping test, it is now possible to start an SSH session using the following command:

```
1 ssh fio@portenta-x8-182
```

The example command above starts an SSH (Secure Shell) connection to the Portenta X8 with the hostname

```
portenta-x8-  
1822aa09dab6fad9.local
```

using the username `fio`. The command format should be as follows:

```
1 ssh fio@portenta-x8-<UU
```

When executing the command, substitute the `<UUID>` placeholder with the actual UUID of the Portenta X8 you are attempting to connect to. You can confirm this UUID by checking the results of a prior SSH services scan with a network query or ADB shell.

If the device is configured correctly to accept SSH connections and the `fio` account exists with SSH access, this command will prompt for the password associated with the `fio` user.

the device, allowing for command-line interface access and the execution of commands remotely on the Portenta X8.

The password and the rest of the configuration for using the Portenta X8 inside the shell remain the same.

The process is similar for *GNU/Linux* and *macOS*, with minor differences in the initial steps when browsing for SSH services on the local network.

For GNU/Linux:

Use *Avahi-Browse* to search for SSH services on the local network:

```
1 avahi-browse -d local _
```

macOS:

On macOS, you can use the similar command:

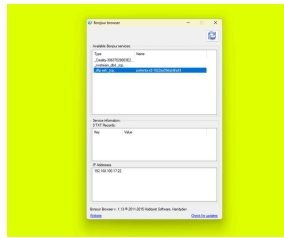
```
1 dns-sd -B _sftp-ssh._tcp
```

Alternatively, you can use a software called *Discovery*, which is available [here](#).

Using Software With GUI

The SSH session can be initialized using third-party software with a Graphical User Interface (GUI) for easy access. An example is a

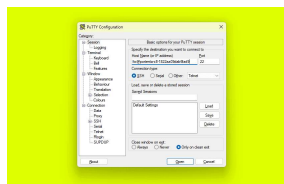
software called *Bonjour Browser*, which can be downloaded [here](#).



SSH Services Availability Discovery with GUI

This software simplifies browsing SSH services on the local network advertised over mDNS using a GUI. The image above, for example, shows all available services on the network, including those for the Portenta X8. You can retrieve the IP address information by simply clicking on a service item.

Once the information is verified, you can use that data with software such as *PuTTY*. *PuTTY* is a free and open-source terminal emulator, serial console, and network file transfer application. It supports several network protocols, including *SSH (Secure Shell)* and *SFTP (SSH File Transfer Protocol)*.

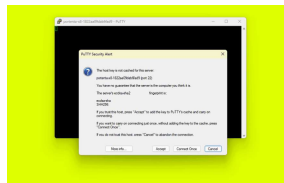


Portenta X8 SSH Session with PuTTY - Setup

values, you must specify the *Host Name (or IP address)* field with `portenta-x8-<UUID>`. For instance, you would use:

```
1 portenta-x8-1822aa09dab
```

Click on `Open`, and it will prompt a security alert. It displays information about the connection, including fingerprint details. Depending on your connection profile preference, you can choose to `Accept` or `Connect Once`.



Portenta X8 SSH
Session with PuTTY -
Authentication

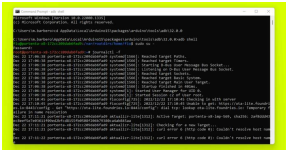
After verifying the security alert and proceeding, you have an SSH session that has begun communicating with the Portenta X8.



Portenta X8 SSH
Session with PuTTY

Inspect Real-Time Tasks And Logs Via CLI

Run `journalctl -f` to check the status of active services and possibly their errors, as well as various system event logs.



ADB shell journalctl package

By calling `journalctl`, it is possible to look at the log of all the running activities by specifying the type of log you are looking for. Some logs may be a warning that can be ignored, while some may be critical errors. Type `journalctl -p 0` to view emergency system messages; otherwise, change the number 0 with the error code you want to investigate according to the following error code numbers:

Error code	Meaning
0	Emergency
1	Alerts
2	Critical
3	Errors
4	Warning
5	Notice
6	Info
7	Debug

When you specify the error code, it shows all messages from that code and above. For example, if you specify error code 2, it shows all messages with priority 2, 1 and

You can also view logs for a specific time and date duration. You can use the `-- since` switch with a combination of `"yesterday"`, `"now"`, or a specific date and time `"YYYY-MM-DD HH:MM:SS"`.

An example of how to use the command:

```
1 journalctl --since "202
```

Working with Docker on Portenta X8

To customize the Portenta X8, you can develop and deploy Docker containers. Start by ensuring Docker is correctly set up on your device. A quick way to test Docker functionality is by running the official `Hello World` container from [Docker Hub](#).

First, pull the image using the following command:

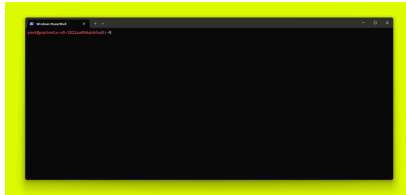
```
1 docker pull hello-world
```

Then, run the container:

```
1 docker run hello-world
```

This simple process pulls and runs

verifying that Docker is available and functional on your Portenta X8. The following clip shows running the `Hello World` container briefly:



With this, you can start customizing the Portenta X8 by creating and deploying containers tailored to your needs. For example, you can develop Python applications or other utilities and package them into Docker containers for deployment.

You can explore [Arduino's Docker Hub](#) for a



repository of preconfigured Docker images designed to simplify development on the Portenta X8.

Build Your Container

You can create your custom containers and build them inside the Portenta X8. Since Portenta X8 is based on an *arm64 architecture*, you can use the command `build` only if you build the container directly on an *arm64 architecture* (e.g., Macbook based on M1/M2 processor or Portenta X8). Open a terminal window and type:

```
1 docker build . -t x8-cu
```

Otherwise, if you are using a different architecture or building machine, use the `buildx` command to specify which architecture your build should compile for:

```
1 docker buildx build --p
```

This way, your Docker image will be built and tagged with the name `x8-custom-devel`.

It is time for you to deploy the newly created Docker image. To do so, save it somewhere and deploy it on your Portenta X8.

Managing Early Start Services When Building Custom Containers

The Portenta X8 firmware includes the

```
compose-apps-early-  
start.service
```

, which starts certain Docker Compose applications early during the boot process. This feature helps pre-configured services run smoothly but may sometimes interfere with custom containers you pull or build.

For example, system tools like **Skopeo** may automatically

containers pulled from external sources or locally built on the device. If you notice that your custom containers are being removed unexpectedly, you can solve this by managing the system services with a few command lines.

To prevent automatic container removal and ensure your custom containers stay intact, the early start services can be stopped and disabled by running the following commands in the ADB shell:

```
1 systemctl stop compose-  
2 systemctl stop compose-  
3 systemctl disable compo  
4 systemctl disable compo
```

Alternatively, you can use this single line of command:

```
1 systemctl stop compose-
```

Stopping and disabling these services will prevent the early start of compose applications, ensuring your custom containers are not removed automatically. Additionally, make sure to check for the [latest firmware image](#) to maintain compatibility and optimal performance of the Portenta X8 with custom container developments.

If you have a [Docker Hub account](#), you can freely upload your Docker image to your registry (e.g., `yourhubusername`):

```
1 docker push yourhubuser
```

Your image is now available in your Docker Hub registry `yourhubusername`.

At this point, you can directly pull the image to your Portenta X8. To do so, connect to your Portenta X8 through ADB. It can be found at

```
Arduino15\packages\arduino\tools\adb\32.0.0
```

You can pull the image from that directory to the preferred location.

```
1 adb shell
```

With the Portenta X8's terminal, the following command is used.

```
1 docker pull x8-custom-d
```

Now, your image is correctly deployed on your Portenta X8.

Deploy Your Container Without Docker Hub

If you do not have a Docker Hub account, you can also save the Docker container locally as a **.tar** file. Then, you can easily load that to an image.

You can use the 'docker save' command to save a Docker image after you have built it. For example, let's save a local copy of the `x8-custom-devel` docker image you made:

```
1 docker save x8-custom-d
```

At this point, you can directly pull the image to your Portenta X8. To do so, connect to your Portenta X8 through ADB. It can be found at

```
Arduino15\packages\arduino\tools\adb\32.0.0
```

```
1 docker import /home/fio
```

Now, your image is correctly deployed on your Portenta X8.

Launch Your Container

To launch your brand new image, you need to create a new `docker-compose.yml`. To do so, first, you must stop the current `docker-compose.yml`.

```
1 cd /var/sota/compose-ap
```

You can now create the path for the new `docker-compose.yml`:

```
1 mkdir /var/sota/compose
```

Before uploading, open the `docker-compose.yml` and edit it as follows to make it use the Docker image you have just created:

```
1 services:
2   custom:
3     container_name: cust
4     hostname: "portenta-
5     image: x8-custom-dev
6
7     restart: unless-stop
8     tty: true
9     read_only: false
10    user: "0"
11    volumes:
12      #- '/dev:/dev'
13      - '/run/arduino_hw
14      - '/sys/devices:/s
15      - '/sys/class/pwm:
16      - '/sys/bus/iio:/s
17      - '/var/sota:/var/
18      - './keys:/tmp/key
19    devices:
20      - '/dev/gpiochip5'
21      - '/dev/tee0'
```

It is now time to upload the new `docker-compose.yml` to your Portenta X8:

```
1 docker compose up --det
```

And you are ready to go! Your Portenta X8 Dashboards and Things can be customized using the same process multiple times.

If you are using the Portenta X8 Manager, go to [this documentation](#) to learn how to upload the newly created container in your FoundriesFactory.

In-Depth Docker Container Management On Portenta X8

We created dedicated tutorials covering this topic. Go check them out:

[Managing Containers with Docker on Portenta X8](#)

[Deploy a Custom Container with Portenta X8 Manager](#)

[Running Wordpress & Database Containers on Portenta X8](#)

Output Video Content On A Screen

The USB-C® port on your Portenta X8 supports video output. Consequently, you can connect a USB-C® monitor or a USB-C® to the HDMI hub to your Portenta X8 to start visualizing video or other visual renders.

HDMI hubs:

[TPX00145](#)

[TPX00146](#)

 Learn more on how to output WebGL content on a screen with Portenta X8 by checking the [dedicated tutorial](#).

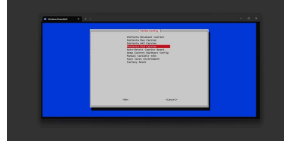
Build A Custom Image For Portenta X8

You may want to build a custom image for the Portenta X8 with the source code provided in the public [GitHub repository of Imp-manifest](#). Building an image locally can help debug certain aspects of the system, such as the bootloader or kernel support.

 Have a look at [this dedicated tutorial](#) to understand how to build your own custom image.

Using the Integrated Overlay Configurator

tenta-config is a configuration tool for managing hardware settings and device tree overlays on compatible carriers. It provides a graphical interface for customizing system configurations with compatible carriers, such as peripheral management, adjusting video output settings, and managing pin mappings.



Portenta X8 tenta-config Main Screen

The image above shows the main window when accessing the `tenta-config`. It lists all compatible carriers, the option to automatically probe the attached carriers, and additional operations for debugging and configuring purposes for the Portneta X8.

This tool simplifies hardware customization, allowing users to modify their setup without modifying the core device tree, which is essential for tasks like configuring display settings or activating specific hardware features.

To access the `tenta-config` window, please follow these instructions.

First, access the Docker container named **x8-devel** with the following command:

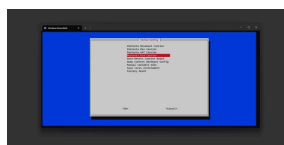
```
1 docker exec -it x8-devel
```

This command uses **docker exec** to start a new shell session inside the running **x8-devel** container. The `-it` flags provide an interactive terminal session for executing commands within the

```
1 find / -name *.py
```

```
1 cd /root/examples/tenta
```

```
1 python tenta_runner.py
```



Portenta X8 tenta-

If you have a Pro 4G Module or a GIGA Display Shield with the Portenta Mid Carrier, you can choose the

`Portenta Mid Carrier` option and set the necessary overlays to prepare essential environment configuration with a few steps.

You can find more information on how to use the `tanta-config` to set up the Pro 4G Module [here](#) and the GIGA Display Shield [here](#).

Additional Tutorials

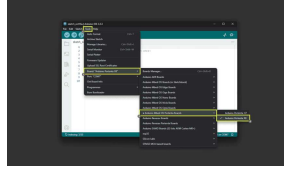
If you want to continue working with your Portenta X8, you can find tons of additional tutorials in the **Tutorials** section of our [Arduino Docs](#). Please go check them out!

Working With Arduino

Working With Arduino IDE

In this section, you will learn how to upload a sketch to the M4 core on the STM32H747XI MCU.

Open the Arduino IDE and download the latest **Arduino Mbed OS Portenta Boards Core** following [this tutorial](#). Then, select **Portenta X8** from the board selector.



IDE Board Selector

Create a custom sketch or open an example like the blink sketch:

```
1 void setup(){
2   pinMode(LED_BUILTIN ,
3 }
4
5 void loop(){
6   digitalWrite(LED_BUIL
7   delay(1000);
8   digitalWrite(LED_BUIL
9   delay(1000);
10 }
```

Select the device port and press **Compile and Upload**. The sketch is compiled into a binary, uploaded to the Linux side of the Portenta X8, and flashed by the RPC service running on Linux.

Please refer to the [Communication Between Arduino and Linux](#) section for details on how Remote Procedure Call enables this communication.

Once the upload is successful, the onboard LED of your Portenta X8 will blink at a one second interval.

If you prefer to upload the firmware manually, first compile the sketch by selecting **Export compiled binary** from the Sketch

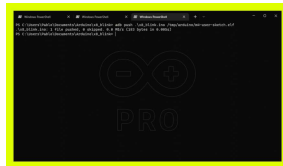
binary file in the sketch folder. Alternatively, you can use the [Arduino CLI](#) to create an `elf` file.

You can use the ADB tool installed as part of the Portenta X8 core to upload the firmware. It can be found at

```
Arduino15\packages\arduino\tools\adb\32.0.0
```

From that directory, you can use the `adb` tool. To upload your compiled sketch, you need to type the following command into your terminal window:

```
1 adb push <sketchBinaryP
```



ADB upload with a terminal

Communication Between Arduino and Linux

The Portenta X8 uses **RPC (Remote Procedure Call)** to exchange data between processors, allowing remote **procedures** or **functions** to run over a network while appearing as local calls.

In a client-server model, **RPC**

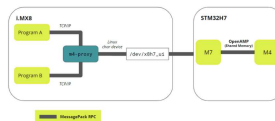
client **requests** and **return values** as server **responses**. To ensure reliability, two semantics are used:

At most once : Prevents duplicate execution if a failure occurs.

At least once : Guarantees at least one execution, even if repeated.

The Portenta X8 relies on **MessagePack-RPC** for communication, supporting *OpenAMP*, *SPI*, *Linux Char Device*, and *TCP/IP*.

The **M7 core** of the **STM32H7** manages data flow between the Linux and Arduino environments, working as an intermediary when the **M4 core** runs an Arduino sketch. At the same time, **m4-proxy** eases data transfer on the Linux side. This design does not support traditional dual-core processing.



Linux Arduino RPC



RPC M4 proxy

You can make interesting systems with the Arduino environment by leveraging the RPC communication between the Arduino and Linux layers. Check

examples of using RPC with the Portenta X8.

For further details about Remote Procedure Call and its implementation, please refer to [this dedicated tutorial](#).

Advanced Arduino-Linux Integration

Build Arduino sketches for real-time tasks like sensor communication and Fieldbus management. Then, send data to the cloud or remote servers using Linux's networking capabilities.

For example, the [Data Exchange Between Python® on Linux and an Arduino Sketch](#) tutorial will help you understand how to exchange sensor data between the Python® container embedded on Portenta X8 and an Arduino sketch.

If you are a more advanced user, you can check [Multi-Protocol Gateway With Portenta X8 & Max Carrier](#) tutorial on developing your multi-protocol gateway. It aims to receive data from a sensor with the Arduino layer via MQTT protocol, take advantage of RPC to establish communication between Arduino and Linux, and then send the acquired data to *The Things Network* via *LoRa®-based network* managed by the Linux layer.

Working With Portenta X8 Board Manager

The Portenta X8 Board Manager helps you keep your Portenta X8 Linux image and containers up to date, even remotely, using Over-The-Air (OTA) updates via wireless connectivity.

An **Arduino Cloud for Business** plan with a Portenta X8 Board Manager subscription is required to enable these features.

Using the Portenta X8 Board Manager is optional and not required for the Portenta X8 to function. It

leverages Foundries.io's infrastructure to enable

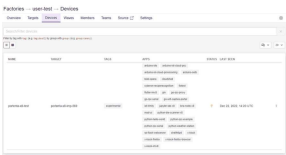


features like secure Over-The-Air (OTA) updates for the Portenta X8 OS and applications. **While it simplifies development and management processes, it is not a mandatory component.**

For more information on configuration and environment exploration, please refer to the [Portenta X8 Board Manager](#) section at the beginning of the user manual.

Device And Fleet Management With Portenta X8 Board Manager

Verify that your Portenta X8 is correctly added to your FoundriesFactory by checking if it is listed among the available devices under the **Devices** section.



FoundriesFactory
device overview

If you want to check if your Portenta X8 is updated according to the latest available Target (i.e., update), you can check the bulb's color under the status column. There are three main color options:

Bulb color	Meaning
Green	Device online and updated
Yellow	Device online and not updated
Red	Device offline and not updated

In this case, the Portenta X8 is connected to the network (and to the FoundriesFactory) but has not been updated.

You can see the Target uploaded on your device under the Target column, i.e., *portenta-x8-1mp-569*, and get additional information about what is included in this specific Target by clicking on it.



FoundriesFactory
device target specs

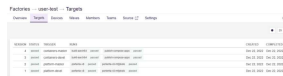
The above window also shows you all the container apps you have installed on your device, and you can start using them.

If you scroll down in the same window, you can also look at the update history of that specific device.



FoundriesFactory
update history

You can now compare the Target uploaded on your Portenta X8 with the Target available in the **Targets** section and decide whether to update your device.



FoundriesFactory target
overview

Learn how to update your Portenta X8 with your FoundriesFactory by checking the [dedicated section](#) of this user manual.

This **Target** page contains the Linux images built each time something is committed in the

Source section. In this section, you can find the four repositories that are used to customize the images:

ci-scripts.git: Scripts that define the platform and container build jobs on the FoundriesFactory system.

containers.git: This is where containers and docker-compose apps are defined. It allows you to define which containers to build/deploy and how to orchestrate them on the platform.

Imp-manifest.git: The repo manifest for the platform build. It defines which layer versions are included in the platform image. This includes **meta-partner-arduino**, the layer containing Arduino-specific customizations (machine definition, device drivers, etc.).

meta-subscriber-overrides.git: *OE* layer that defines what is included in your FoundriesFactory image. You can add board-specific customizations and overrides or add and remove packages provided in the default Linux microPlatform.

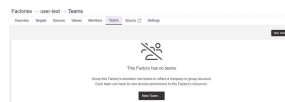
Committing to **Imp-manifest.git** or **meta-subscriber-overrides.git** repositories will create a platform Target, i.e. base Linux platform image. On the other hand, committing to **containers.git** will create a container Target, including all the containers and docker-compose apps you would like to upload on your Portenta X8

Both these Targets will generate the artifacts specified in the **ci-scripts.git**, which includes all the required files to program the Target in case of platform build.

RBAC With Portenta X8 Board Manager

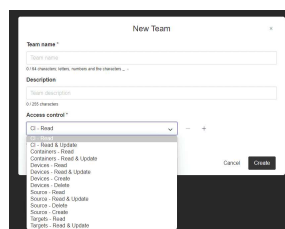
You do not have to be the only one in your organization with permission to update your Portenta X8 devices. The FoundriesFactory integrates a Role-Based-Access-Control functionality (RBAC) to allow users to add multiple teams with multiple members each.

You can start defining a new team by clicking the **Teams** section.



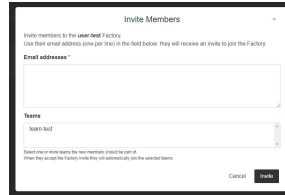
FoundriesFactory
Teams

The level of access and specific permissions are defined by the team's role in the FoundriesFactory. As you can notice from the image below, multiple roles and permissions are available.



FoundriesFactory Team roles and permissions

Once you created the team, you can go to the **Members** section of your FoundriesFactory to invite new members to the team.



FoundriesFactory new member

You can type the email addresses of your teammates, and they will receive an automatic email with the invitation to join the corresponding team in your FoundriesFactory.

FoundriesFactory FIOCTL

The FoundriesFactory includes a command line tool called **FIOCTL**, which allows you to manage your Portenta X8 through your CLI.

With this tool, you can easily upload containers to a board linked to your FoundriesFactory just by stating the FoundriesFactory name, the board name, and the app you would like to upload.

Learn how to use this tool by checking the dedicated tutorial at [this link](#) or the corresponding [Foundries documentation](#).



Pins

In order to learn how to properly call GPIOs or other peripherals both in the Arduino environment or in Linux, with or without a carrier, you can check the following pinout diagrams:

[Portenta X8 pinout](#)

[Portenta Breakout pinout](#)

[Portenta Max Carrier pinout](#)

[Portenta Hat Carrier pinout](#)

[Portenta Mid Carrier pinout](#)

Communication

In this section, you will learn how to make your Portenta X8 communicate with multiple types of sensors or other external devices, leveraging the wide variety of supported interfaces:

[SPI](#)

[I2C](#)

[UART](#)

[Bluetooth®](#)

SPI

In this case, a Portenta X8 with a Portenta Breakout board is used to connect an external SPI device.

SPI With Linux

You need to enable SPI support before using SPI devices.

With root privileges on the Portenta X8, you can enable the SPI device interface by loading the `spidev` module using the following command:

```
1 sudo modprobe spidev
```

To ensure the `spidev` module loads automatically at startup, add it to the system's module configuration and reboot:

```
1 echo "spidev" | sudo te
2 sudo systemctl reboot
```

To set up a service named `my_spi_service` that utilizes the SPI device at `/dev/spidev0.0`, include the following configuration in your service definition:

```
1 services:
2   my_spi_service:
3     devices:
4       - "/dev/spidev0.0"
```

This configuration is typically added to a `docker-compose.yml` file, allowing the service to interact with the specified SPI device.

If the Linux user on which the container is running is not `root`,

the SPI devices. You might add the required comments to an `entrypoint.sh` shell file (to be added to the `Dockerfile` or the `docker-compose.yml` file).

```
1 #!/usr/bin/env sh
2
3 # entrypoint.sh example
4
5 chgrp users /dev/spi-*
6 chmod g+rw /dev/spi-*
7 usermod -aG users <container user>
8
9 # Possible command to
10 # Check https://github.com
11 gosu <container user>
```

SPI Port Mapping

Linux	Arduino Portenta Breakout
134	SPI1 CK
135	SPI1 COPI
136	SPI1 CIP0
137	SPI1 CS

SPI With Arduino

The `SPI` object is **mapped** as follows on the Portenta Breakout Board and can be deployed as usual:

SPI Pin	Arduino Portenta Breakout
CIP0	Pin 0 (Header GPIO0)
COPI	Pin A6 (Header Analog)
SCK	Pin A5 (Header Analog)

I2C

In this case, a Portenta X8 with a Portenta Breakout board is used to connect an external I2C device.

I2C With Linux

You need to enable I2C support before using I2C devices.

Open Portenta X8 Shell as explained [here](#).

With administrative (root) privileges on the Portenta X8, you can enable the I2C device interface using the following command:

```
1 sudo modprobe i2c-dev
```

Use the following commands to ensure the `i2c-dev` module is automatically loaded at system startup. A system reboot is required for the changes to take effect:

```
1 echo "i2c-dev" | sudo t
```

```
1 sudo systemctl reboot
```

These commands activate and configure the I2C device interface

up I2C services within a Docker environment by defining them under `my_i2c_service` in a `docker-compose.yml` file:

```
1 services:
2   my_i2c_service:
3     devices:
4       - "/dev/i2c-0"
5       - "/dev/i2c-1"
6       - "/dev/i2c-2"
7       - "/dev/i2c-3"
```

You can configure I2C services for all available interfaces (from 0 to 3) or specify particular services as needed.

If the Linux user on which the container is running is not `root`, you need to set up the permissions for the user to access the I2C devices. You might add the required comments to an `entrypoint.sh` shell file (to be added to the `Dockerfile` or the `docker-compose.yml` file).

```
1 #!/usr/bin/env sh
2
3 # entrypoint.sh example
4
5 chgrp users /dev/i2c-*
6 chmod g+rw /dev/i2c-*
7 usermod -aG users <container user>
8
9 # Possible command to
10 # Check https://github.com/tianon/gosu
11 gosu <container user>
```

Linux	Arduino Portenta Breakout	Notes
/dev/i2c-1	I2C1	
/dev/i2c-2	I2C0	Recommended
/dev/i2c-3	I2C2	

Examples

Within the Portenta X8 shell, you can quickly test I2C communication with compatible devices using specific commands. To list all connected I2C devices, use:

1 i2cdetect -y <I2C bus>

To interact with a specific I2C device and retrieve data, the command format is:

1 i2cget -y <I2C bus> <de

For example:


```
1 # List I2C devices on b
2 i2cdetect -y 2
3
4 # Read data from a devi
5 i2cget -y 2 0x77 0xD0
```

If a BME280 sensor is connected to `I2C0` and the above commands are used, the Portenta X8 will show `0x60` value to tell the sensor is alive.

Here are some simple examples of implementing I2C communication on the Portenta X8 with Portenta Mid Carrier using Python.

The following code uses the SMBus (System Management Bus) protocol, with SMBus-compatible [libraries](#), to read a data byte from a device on `/dev/i2c-3`. The byte is read from address 80, offset 0, and then printed.

Examples of using SMBus-compatible [libraries](#):

```
1 from smbus2 import SMBu
2
3 # Connect to /dev/i2c-3
4 bus = SMBus(3)
5 b = bus.read_byte_data(
6 print(b)
```

The next example shows how to initialize the I2C bus using the `smbus2` library and read multiple bytes from a device. The `read_i2c_block_data` function retrieves a block of bytes from the

```
1 from smbus2 import SMB
2
3 # Initialize the I2C b
4 bus = SMBus(3) # 3 re
5
6 device_address = 0x1
7 num_bytes = 2
8
9 # Read from the I2C de
10 data = bus.read_i2c_bl
11
12 # Data is a list of by
13 for byte in data:
14     print(byte)
```

The following example shows how to write data to an I2C device using the *smbus2* library. A data byte (`value`) is sent to a specific address (`device_address`) with a given instruction.

```
1 from smbus2 import SMB
2
3 # Initialize the I2C b
4 bus = SMBus(3) # 3 re
5
6 device_address = 0x1
7 instruction = 0x00
8 value = 0xFF
9
10 # Write to the I2C dev
11 bus.write_byte_data(de
```

In the following example, the [python-periphery](#) library is used to communicate with the I2C device. This library integrates a broad range of protocols within the same script. The `I2C.transfer()` method

The code reads a byte from an EEPROM at address `0x50`, offsets `0x100`, and then prints it.

```
1 from peripheral import
2
3 # Open i2c-3 controller
4 i2c = I2C("/dev/i2c-3")
5
6 # Read byte at address
7 msgs = [I2C.Message([0
8 i2c.transfer(0x50, msg
9 print("0x100: 0x{:02x}
10
11 i2c.close()
```

I2C With Arduino

The `Wire` object is mapped to pins `PWM6` (I2C `SCL`) and `PWM8` (I2C `SDA`) on the Portenta Breakout Board and can be deployed as usual.

Since one of the `I2C` pins is GPIO-multiplexed, you need to detach it from the other GPIO. Just add the following line to the `setup()` definition to make it fully functional for `I2C` operations.

```
1 void setup() {
2   pinMode(PA_12, INPUT);
3 }
```

UART

In this case, a Portenta X8 with a

communication. The Portenta Mid Carrier and Portenta Hat Carrier also provides the capability to use the UART communication.

UART With Linux

A standard UART is available as `/dev/ttymx1` in Linux and is mapped to the `UART1` port on the Portenta Breakout.

With root access on the Portenta X8, you can list the available serial ports in Linux using the following command:

```
1 ls /dev/ttyUSB* /dev/tty
2
3 // Example output:
4 /dev/ttyUSB0 /dev/tty
5 /dev/ttyUSB1 /dev/tty
```

Common serial devices are typically labeled as `/dev/ttyUSBx`, `/dev/ttyACMx`, or `/dev/ttymx`. For instance, you might see `/dev/ttymx1` listed.

The following Python script uses the *pyserial* library for UART communication. The *processData* function is defined to handle incoming data, which can be customized according to your needs.

```
1 import serial
2 import time
3
4 # Define the processData function
5 def processData(data):
6     print("Received:" + data)
7
8 # Set up the serial port
9 ser = serial.Serial('/dev/ttyMXC1', 9600)
10
11 incoming = ""
12
13 while True:
14     # Check for available data
15     while ser.in_waiting:
16         c = ser.read(1)
17
18         # Check if the character is a newline
19         if c == '\n':
20             # Process the data
21             processData(incoming)
22
23             # Clear the incoming buffer
24             incoming = ""
25         else:
26             # Add the character to the incoming buffer
27             incoming += c
28
```

This script maintains a serial connection on `/dev/ttyMXC1` with a baud rate of 9600. It reads incoming data character by character until it encounters a newline (`\n`), which indicates the end of a data packet.

After processing the data, the script resets and is ready for the next message. The `time.sleep(0.002)` introduces a brief delay for data buffering, similar to `delay(2);` in the Arduino environment.

Bluetooth®

Portenta X8 supports Bluetooth®

side.

In order to communicate with Bluetooth® devices via the Portenta X8 Shell, you can use the Bluetooth® utility **bluetoothctl**. These are some of the most used commands:

```
bluetoothctl devices
```

to list all the available Bluetooth® devices

```
bluetoothctl pair  
[mac_address]
```

to pair with a specific device through its MAC address

```
bluetoothctl connect  
[mac_address]
```

to connect to a paired device

```
bluetoothctl disconnect  
[mac_address]
```

to disconnect from a paired device



Do you want to send data from your Nicla to Portenta X8 via BLE? Check [this link](#) to get started.

Support

If you encounter any issues or have questions while working with the Portenta X8, we provide various support resources to help you find answers and solutions.

Help Center

Explore our [Help Center](#), which offers a comprehensive collection of articles and guides for the Portenta X8. The Arduino Help Center is designed to provide in-

help you make the most of your device.

[Portenta Family help center page](#)

Forum

Join our community forum to connect with other Portenta X8 users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta X8.

[Portenta X8 category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Portenta X8.

[Contact us page](#)

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

n is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

